# KIẾN TRÚC HỆ THỐNG

Hệ thống ghép cặp CV – Việc làm theo section
(Section-Aware CV–Job Matching) kèm hàng đợi xử lý file nền

Nhóm nghiên cứu: .....
Giáo viên hướng dẫn: .....

October 21, 2026

**Abstract**

Tài liệu mô tả kiến trúc phần mềm của hệ thống ghép cặp "sơ yếu lý lịch (CV) – việc làm (JD)" theo từng *phần (section)* của văn bản, gồm hai tiến trình lớn: (1) **AI service** viết bằng Python/FastAPI, đảm nhận trích xuất văn bản, phân tích bằng mô hình ngôn ngữ lớn (LLM), sinh embedding theo section, lưu vector và xếp hạng hybrid (semantic + từ khóa + ràng buộc có cấu trúc); và (2) **API chính** viết bằng NestJS, đảm nhận nghiệp vụ người dùng và *gọi* AI service. Trọng tâm của tài liệu là **hàng đợi xử lý file nền** (background processing queue) cho phép người dùng upload CV và nhận trạng thái xử lý từng bước thay cho lời gọi đồng bộ "treo" hàng chục giây. Tài liệu gồm: sơ đồ tổng thể, sơ đồ lớp module, máy trạng thái job, sơ đồ chuỗi lời gọi (bước upload bất đồng bộ), mô hình dữ liệu vector theo section, và **luồng ghép cặp (matching) trình bày theo dạng dễ hiểu**: bốn bước từ tìm ứng viên đến trả kết quả, cách tính điểm mức độ hợp, và hai chiều ghép cặp `jd2cv`/`cv2jd`; phần cuối nêu hướng nghiên cứu khoa học đề xuất.

## Contents

**1 Tổng quan hệ thống** **2**

**2 Kiến trúc bên trong AI service** **2**

**3 Hàng đợi xử lý file nền (phần lõi)** **3**
3.1 Vấn đề . . . 3
3.2 Máy trạng thái của job . . . 3
3.3 Luồng bất đồng bộ khi upload CV . . . 4
3.4 Độ tin cậy . . . 4

**4 Mô hình dữ liệu vector theo section** **4**

**5 Luồng ghép cặp CV – việc làm** **6**
5.1 Từ file tải lên đến kết quả ghép cặp . . . 6
5.2 Các bước trong phần ghép cặp . . . 7
5.3 Điểm mức độ hợp được tính thế nào? . . . 7
5.4 Hai chiều ghép cặp . . . 8

**6 Hợp đồng API (tóm tắt)** **8**

**7 Cấu hình & công nghệ** **9**

**8 Kiểm thử & đánh giá** **9**

**9 Đề xuất hướng nghiên cứu khoa học** **10**
9.1 Câu hỏi nghiên cứu . . . 10
9.2 Ma trận thí nghiệm đề xuất . . . 10
9.3 Dữ liệu & quy trình . . . 10

9.4 Lộ trình 12 tuần (đề xuất) . . . . . 11
9.5 Rủi ro & biện pháp . . . . . 11
9.6 Tài liệu tham khảo chính . . . . . 11

**Phụ lục: cấu trúc thư mục liên quan** **11**

# 1 Tổng quan hệ thống

Hệ thống có hai tiến trình chính (hai mã nguồn khác nhau) và một số thành phần hạ tầng:

- **API chính (NestJS, TypeScript)** — backend nghiệp vụ: xác thực, hồ sơ người dùng, tin tuyển dụng, hợp đồng, thanh toán. Là nơi duy nhất được phép đọc/ghi **PostgreSQL** (nguồn dữ liệu gốc).
- **AI service (Python, FastAPI)** — microservice nội bộ, *không* có quyền ghi PostgreSQL: trích xuất PDF/DOCX, phân tích cấu trúc bằng `qwen3:4b` (chạy local qua Ollama), sinh embedding `BAAI/bge-m3`, lưu vector vào **ChromaDB**, xếp hạng và giải thích bằng LLM.
- **ChromaDB** — cơ sở vector (HNSW, cosine) persistent trên đĩa.
- **SQLite** — bảng `processing.db` lưu *trạng thái job xử lý file* (hàng đợi nền). Lý do dùng SQLite: ở giai đoạn này AI service chỉ được cấp quyền *đọc* PostgreSQL; giao diện `ProcessingStore` được thiết kế để sau này thay bằng `PostgresProcessingStore` mà không đổi pipeline.
- **Outbox + `sync_worker`** — cơ chế đồng bộ *một chiều* PostgreSQL → Chroma: các bản ghi CV/JD trong DB được đẩy sang vector store để phục vụ truy hồi.

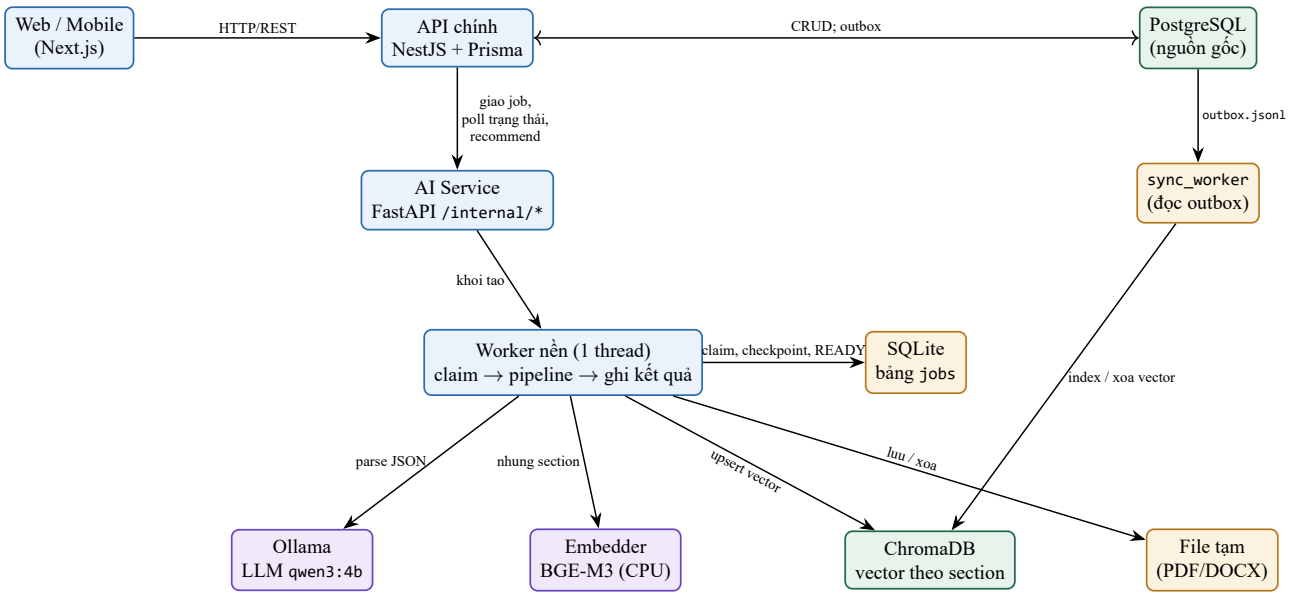



**Figure 1:** Tổng quan hệ thống: API chính (NestJS) điều phối, AI service (FastAPI) xử lý và xếp hạng, dữ liệu vector nằm ở ChromaDB, trạng thái job nằm ở SQLite.

Hai thành phần *giao tiếp* với nhau chỉ qua HTTP nội bộ, được bảo vệ bằng `INTERNAL_API_KEY` (che dấu `/internal/*` khỏi Internet). Nhờ đó AI service có thể thay đổi độc lập (thêm model, thêm queue) mà không ảnh hưởng API chính.

# 2 Kiến trúc bên trong AI service

AI service được chia thành bốn lớp phụ trách rõ ràng (Hình 2), theo nguyên tắc *API chỉ điều phối, nghiệp vụ nằm ở lớp giữa, lớp dưới lo hạ tầng*. Thư viện lõi `src/cvjd` (Python package `cvjd`) độc lập với web framework để có thể chạy benchmark từ dòng lệnh.

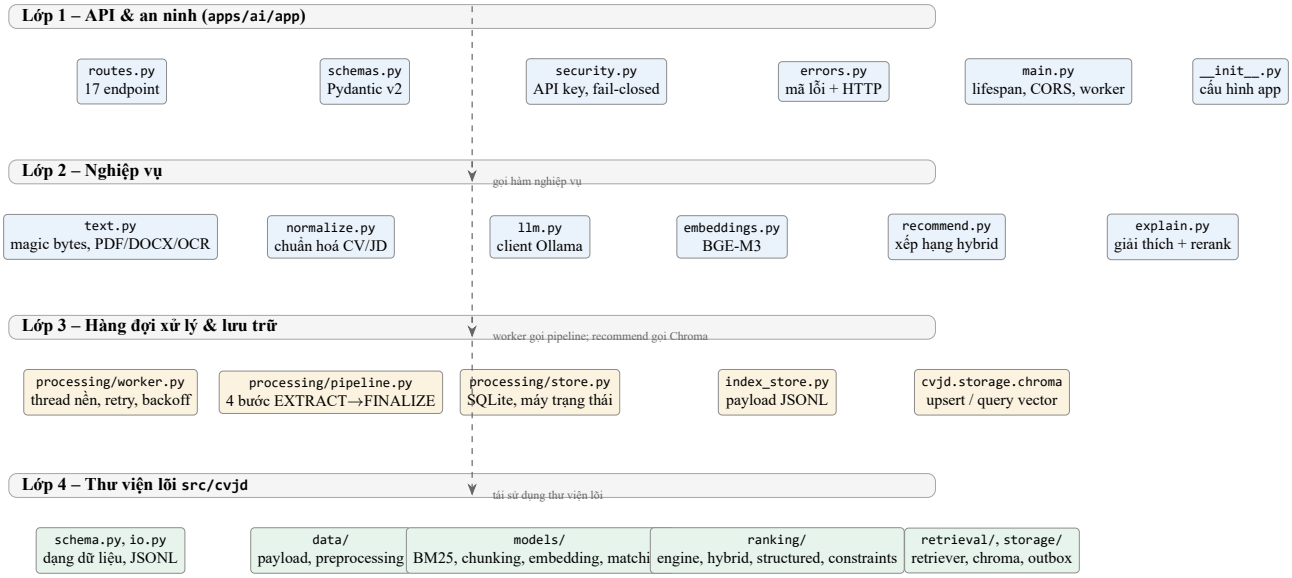



**Figure 2:** Các lớp module của AI service. Lớp ngoài tiếp nhận HTTP, lớp nghiệp vụ xử lý văn bản/LLM, lớp hàng đợi lo việc chạy nền, thư viện lõi chứa các mô hình và thứ tự xếp hạng (có thể chạy riêng từ CLI để benchmark).

Hai đường xử lý "đồng bộ" và "nền" dùng chung các khối nghiệp vụ nên kết quả phân tích là như nhau (Hình 3).

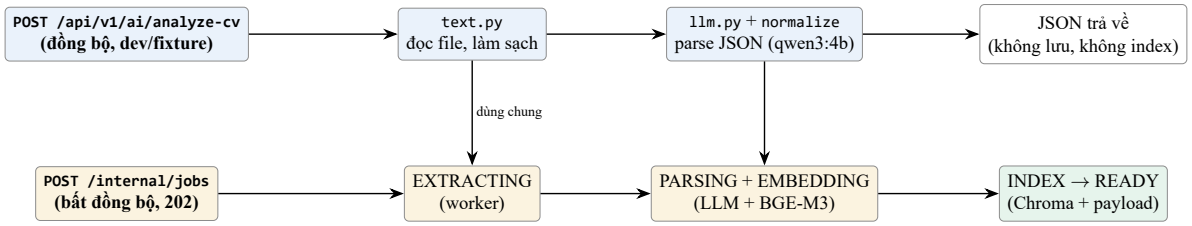



**Figure 3:** Hai đường xử lý dùng chung khối trích xuất và LLM: đường đồng bộ `analyze-cv` chỉ trả JSON, đường hàng đợi còn nhúng và index vector để phục vụ truy hồi sau đó.

# 3 Hàng đợi xử lý file nền (phần lõi)

## 3.1 Vấn đề

Việc phân tích một CV mất 10–40 giây (trích xuất PDF, gọi LLM sinh JSON, nhúng 5–8 section). Nếu để HTTP đồng bộ "treo" chừng đó thời gian thì trình duyệt/người dùng dễ bị timeout và NestJS cũng bị chiếm kết nối. Giải pháp: **hàng đợi job nền**

1. Client gửi file → nhận ngay `202 Accepted` kèm `job_id`;
2. Worker chạy nền xử lý từng job, cập nhật *step* + *progress*;
3. Client poll trạng thái và hiển thị "Đang trích xuất → Đang phân tích → Đang nhúng → Hoàn tất".

## 3.2 Máy trạng thái của job

Mỗi job có *trạng thái* (`status`) và *bước đang chạy* (`step`) là hai chiều độc lập: trạng thái nói "job đang đi tới đâu về mặt vòng đời", bước nói "đang ở khâu nào" để hiển thị tiến độ (Hình 4).

Bên trong `PROCESSING`, pipeline chạy tuần tự bốn bước, mỗi bước ghi *checkpoint* vào DB nên nếu service chết giữa chừng thì ta biết chính xác job đang ở đâu:

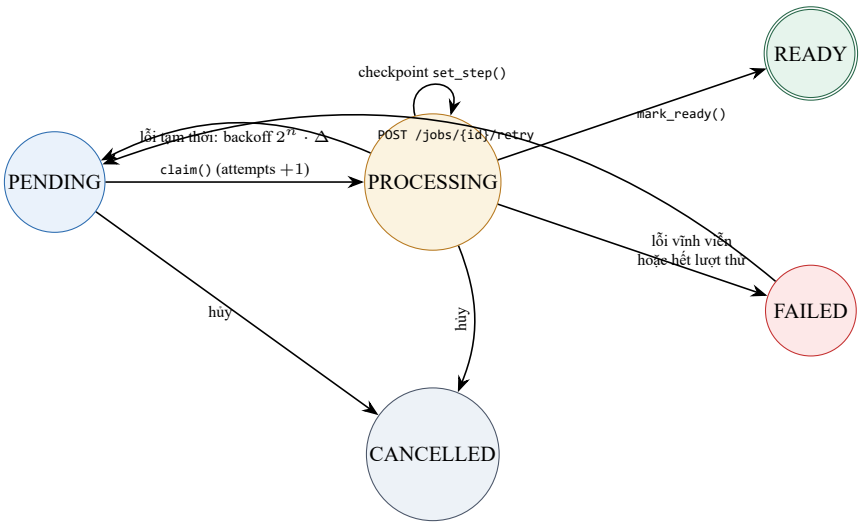



**Figure 4:** Máy trạng thái job xử lý file. `READY` là trạng thái hấp thụ (job hoàn tất); `FAILED` vẫn giữ file gốc để có thể retry thủ công.

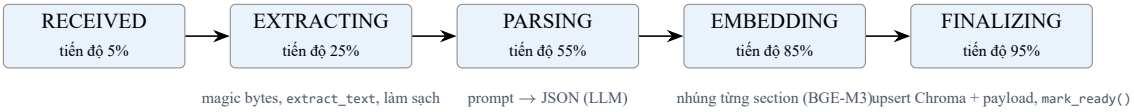



**Figure 5:** Các bước bên trong `PROCESSING` và mức tiến độ tương ứng được ghi vào `step/progress`.

### 3.3 Luồng bất đồng bộ khi upload CV

Hình 6 mô tả toàn bộ lời gọi khi người dùng upload CV: từ HTTP request đến khi job chuyển sang `READY` và client poll được kết quả.

### 3.4 Độ tin cậy

- **Phân loại lỗi:** lỗi *vĩnh viễn* (file hỏng, text rỗng, sai định dạng) → `FAILED` ngay, không thử lại; lỗi *tạm thời* (Ollama timeout, Chroma khóa database) → worker thử lại với backoff $2^n \cdot \Delta$ đến `JOB_MAX_ATTEMPTS` (mặc định 3).
- **Phát hiện job "kẹt":** nếu `PROCESSING` quá `JOB_STALE_SECONDS` (300 s) mà không cập nhật thì worker tự đưa job về `PENDING`; endpoint `POST /internal/jobs/reconcile` cho phép vận hành viên gỡ kẹt thủ công.
- **Không mất job khi restart:** SQLite là nguồn sự thật duy nhất; khi khởi động lại service, các job còn `PENDING` trong DB sẽ được worker xử lý tiếp.
- **Bỏ qua nhúng lại (idempotent):** nếu nội dung text không đổi (`content_hash` khớp) *và* vector của phiên bản đó vẫn còn nguyên trong Chroma + payload còn trong store thì job chuyển thẳng sang `READY` với cảnh báo `REUSED_INDEX` — retry không tiêu tốn thêm lượt LLM.
- **Quan sát được:** `GET /internal/jobs/stats` trả về số job theo trạng thái, độ sâu hàng đợi, số lượt retry và phân vị thời gian xử lý; `GET /readyz` báo trạng thái worker cho giám sát.

## 4 Mô hình dữ liệu vector theo section

Điểm đặc biệt của hệ thống là mỗi tài liệu *không* được nhúng thành một vector duy nhất mà tách theo section (kỹ năng, kinh nghiệm, dự án, học vấn, tóm tắt) cộng thêm một vector cho toàn văn. Nhờ đó khi so một JD với một CV, ta so được "yêu cầu kỹ năng" trùng với "phần kỹ năng" thay vì so toàn văn mơ hồ.

Khi phiên bản tài liệu tăng (`version`+1) thì upsert sẽ tự xoá các vector cũ cùng `document_id` và thay bằng bộ mới; metadata `status=READY` đảm bảo chỉ vector đã hoàn chỉnh mới được dùng cho truy hồi.

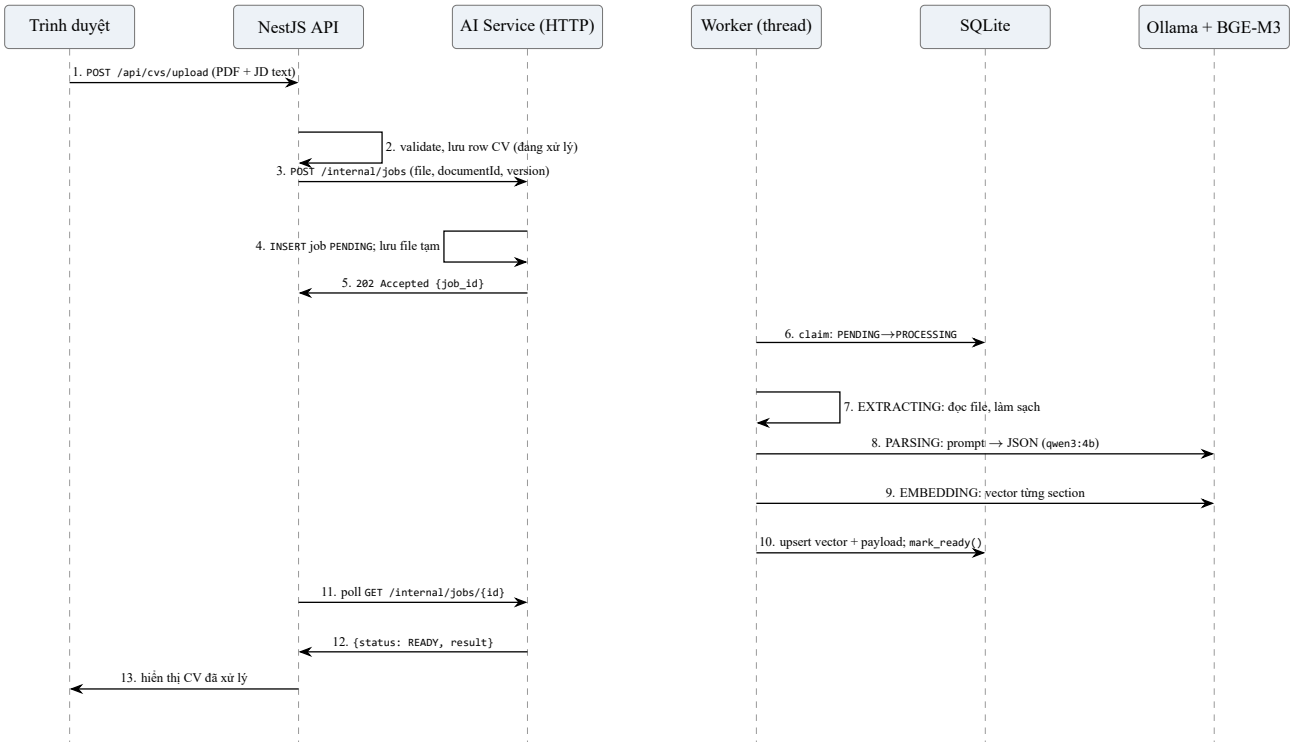



**Figure 6:** Chuỗi lời gọi của luồng upload bất đồng bộ. Toàn bộ bước 6–10 chạy nền trong cùng process FastAPI; API chính chỉ poll trạng thái.

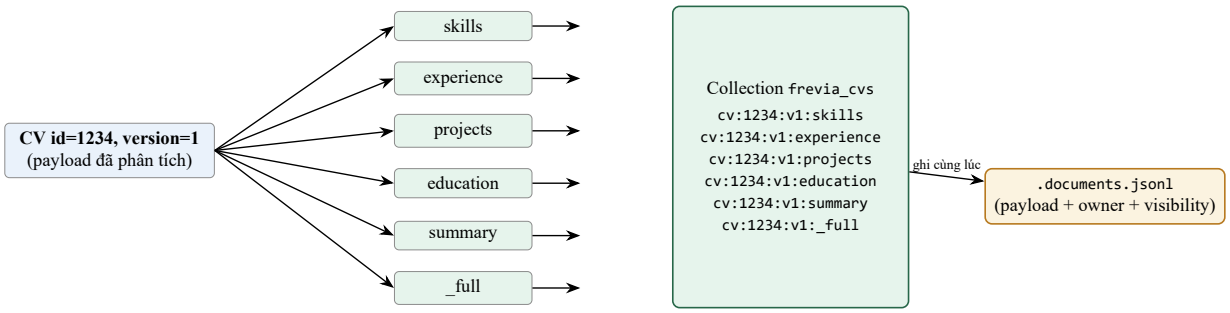



**Figure 7:** Mỗi CV/JD sinh ra một "bó" vector theo section. Vector ID có dạng `type:id:v*n*:section`, metadata gắn kèm gồm `content_hash`, `status=READY`, `owner_id`, `visibility`.

# 5 Luồng ghép cặp CV – việc làm

Đây là phần lõi của hệ thống: cho một người dùng, tìm ra những ứng viên (hoặc tin tuyển dụng) phù hợp nhất.

## 5.1 Từ file tải lên đến kết quả ghép cặp

Trước khi ghép cặp được, tài liệu phải đi qua một chuỗi xử lý để biến văn bản thô thành dữ liệu máy "hiểu" được. Hình 8 mô tả toàn bộ chuỗi đó, gồm tám bước chia làm hai giai đoạn: **xử lý tài liệu** (bước 1–7) và **ghép cặp** (bước 8).

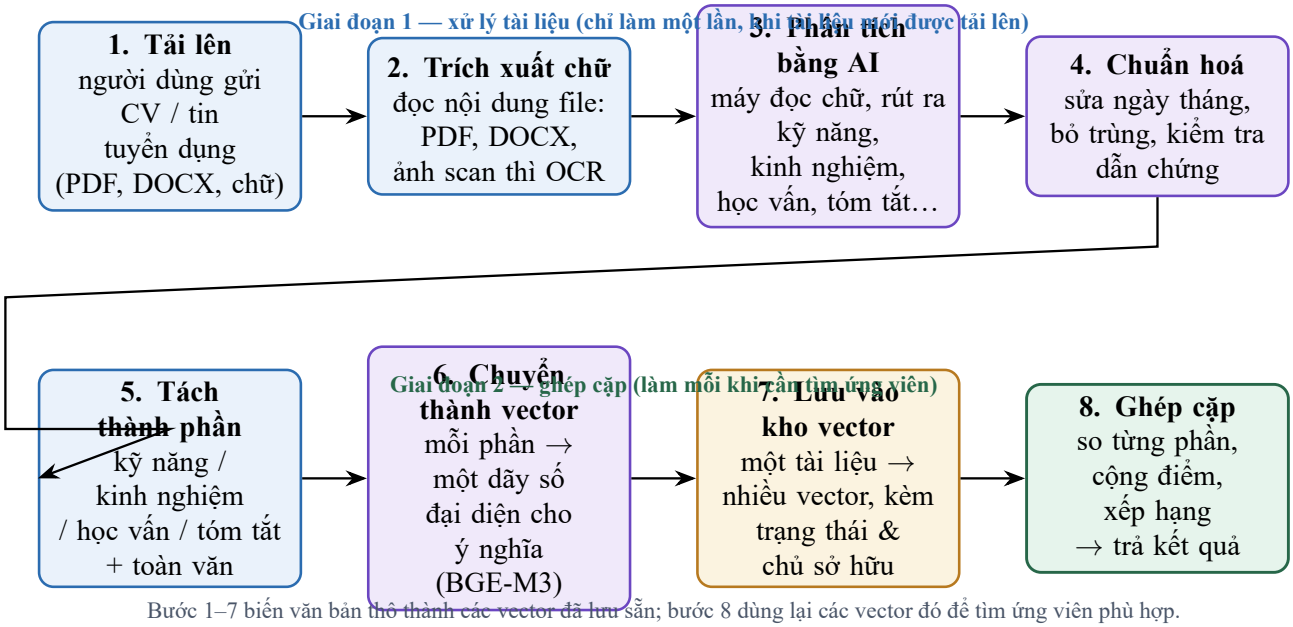



**Figure 8:** Luồng đầy đủ từ khi tài liệu được tải lên đến khi ghép cặp. Giai đoạn 1 (bước 1–7) xử lý và lưu trữ tài liệu một lần; giai đoạn 2 (bước 8) dùng lại dữ liệu đã lưu để ghép cặp và trả kết quả.

Tám bước được giải thích như sau:

1. **Tải lên.** Người dùng gửi một CV hoặc một tin tuyển dụng. Hệ thống nhận file PDF, DOCX (hoặc chữ dán trực tiếp) và lưu tạm lại để xử lý về sau.

2. **Trích xuất chữ.** Hệ thống đọc nội dung chữ ra khỏi file: PDF đọc bằng thư viện `pypdf`, DOCX đọc bằng `python-docx`, file scan (ảnh) thì đọc bằng OCR. Sau đó dọn dẹp ký tự rác, gộp những dòng bị ngắt giữa câu.

3. **Phân tích bằng AI.** Một mô hình ngôn ngữ chạy *nội bộ* (không gửi dữ liệu ra ngoài) đọc đoạn chữ và rút ra thông tin có cấu trúc: kỹ năng, kinh nghiệm, học vấn, dự án, tóm tắt (với CV) hoặc kỹ năng yêu cầu, trách nhiệm, ngân sách (với tin tuyển dụng).

4. **Chuẩn hoá.** Máy sửa lại cho thống nhất: ngày tháng về đúng một dạng, bỏ các mục trùng lặp, và *kiểm tra dẫn chứng* — chỉ giữ những thông tin thật sự xuất hiện trong văn bản gốc.

5. **Tách thành từng phần.** Thay vì coi cả tài liệu là một khối, hệ thống chia thành các *phần* riêng: kỹ năng, kinh nghiệm, học vấn, tóm tắt… và thêm một phần "toàn văn" gộp tất cả.

6. **Chuyển thành vector.** Mỗi phần được mô hình BGE-M3 chuyển thành một dãy số (vector) đại diện cho *ý nghĩa* của phần đó. Nhờ vậy máy so sánh được "giống nhau về nghĩa" chứ không chỉ giống mặt chữ.

7. **Lưu vào kho vector.** Các dãy số được lưu lại, kèm theo trạng thái (đã xử lý xong), chủ sở hữu và mức hiển thị. Một tài liệu sinh ra *nhiều* vector — một cho mỗi phần.

8. **Ghép cặp.** Khi cần tìm ứng viên, hệ thống dùng lại các vector đã lưu, so từng phần với nhau, cộng điểm chi tiết rồi trả về danh sách phù hợp nhất.

## 5.2 Các bước trong phần ghép cặp

Bước 8 ở trên được mở rộng thành bốn bước nhỏ (Hình 9), đi từ trên xuống dưới:

1. **Tìm ứng viên phù hợp.** Nếu bên gọi chưa có danh sách ứng viên, hệ thống tự tìm trong kho dữ liệu đã lưu, xếp theo mức giống nhau về nội dung và chỉ giữ những hồ sơ mà người dùng được phép xem. Nếu bên gọi *đã* có danh sách sẵn thì bỏ qua bước này.
2. **Chấm điểm mức độ hợp.** Hệ thống so nội dung của hai bên *theo từng phần*: phần kỹ năng so với phần kỹ năng, phần kinh nghiệm so với phần kinh nghiệm… thay vì so cả văn bản một cách chung chung. Nhờ vậy điểm số sát ý nghĩa hơn.
3. **Cộng điểm chi tiết.** Ngoài mức hợp về nội dung, hệ thống cộng thêm điểm theo các yếu tố rõ ràng: mức đáp ứng kỹ năng, số năm kinh nghiệm, ngân sách và sở thích (xem Mục 5.4 và 5.3).
4. **Lọc và xếp hạng.** Cuối cùng hệ thống loại những hồ sơ không còn hợp lệ (đã xoá, không đúng người xem, chưa sẵn sàng), sắp xếp theo điểm giảm dần và trả về danh sách phù hợp nhất.

Bước **giải thích bằng AI** là tuỳ chọn: khi bật, máy sẽ tự viết lý do vì sao một ứng viên được xem là hợp, giúp người dùng tin tưởng kết quả hơn.

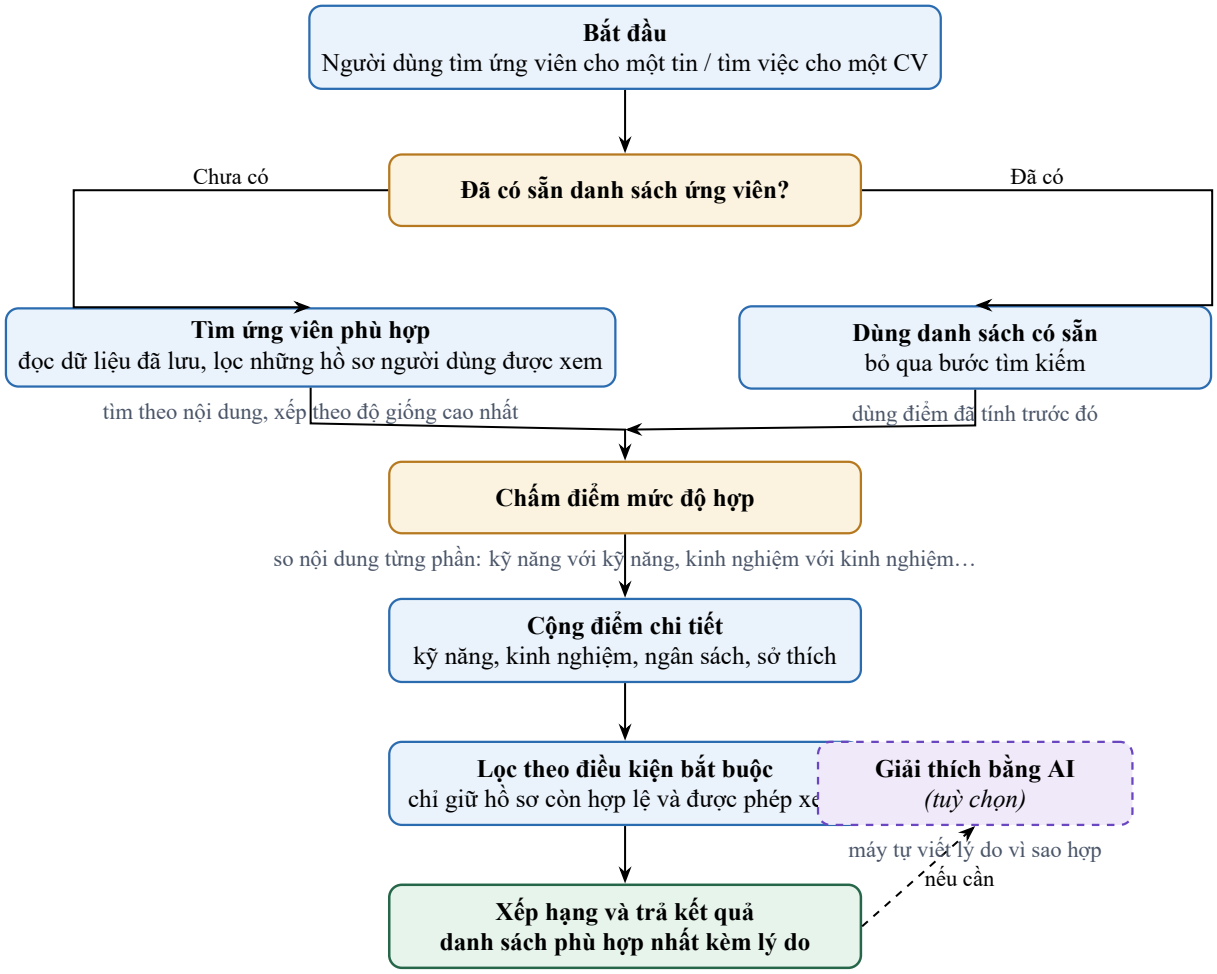



**Figure 9:** Luồng ghép cặp CV–việc làm. Hệ thống tìm ứng viên, chấm điểm mức độ hợp, lọc theo điều kiện bắt buộc rồi trả về danh sách phù hợp nhất; bước giải thích bằng AI là tuỳ chọn.

## 5.3 Điểm mức độ hợp được tính thế nào?

Điểm cuối cùng là trung bình có trọng số của năm phần, mỗi phần nằm trong khoảng 0–1:

| Phần | Trọng số | Ý nghĩa |
|---|---|---|
| Mức hợp nội dung | 0.40 | Hai tài liệu giống nhau đến đâu |
| Kỹ năng | 0.30 | Ứng viên có đủ kỹ năng tin tuyển dụng yêu cầu không |
| Kinh nghiệm | 0.15 | Số năm kinh nghiệm so với yêu cầu |
| Ngân sách | 0.10 | Khoảng lương/ngân sách hai bên có giao nhau không |
| Sở thích | 0.05 | Hình thức làm việc, địa điểm… có khớp không |

Chi tiết công thức:

$$c_k(q,d) = \max_{y \in S_d} \cos\big(e(x_k),\, e(y)\big), \qquad x_k \in S_q \tag{1}$$

$$s_{\text{sem}}(q,d) = \text{agg}_k \; c_k(q,d), \qquad \text{agg} \in \{\max, \text{mean}\} \tag{2}$$

$$s(q,d) = \alpha \cdot \widetilde{s}_{\text{sem}}(q,d) + (1-\alpha) \cdot \widetilde{s}_{\text{kw}}(q,d) \tag{3}$$

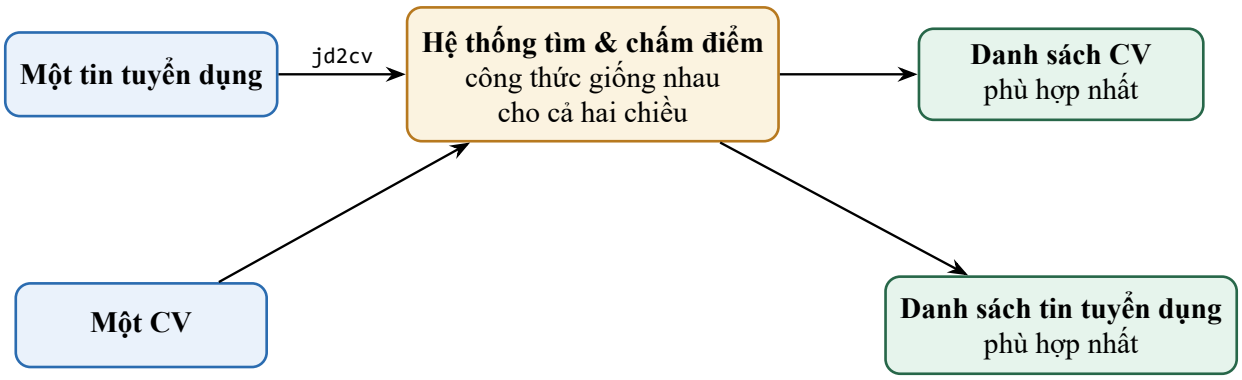
$$S_{\text{hybrid}} = \frac{\sum_i w_i \, S_i}{\sum_i w_i}, \qquad (w_{sem}, w_{sk}, w_{exp}, w_{bud}, w_{pref}) = (0.40,\, 0.30,\, 0.15,\, 0.10,\, 0.05) \tag{4}$$

Trong đó $S_q, S_d$ là tập section của hai tài liệu, $e(\cdot)$ là vector BGE-M3, $c_k$ là điểm của từng phần, $\widetilde{\cdot}$ là chuẩn hoá min–max trên tập ứng viên, và $\alpha$ là tham số pha trộn giữa mức hợp nội dung và mức khớp từ khóa (mặc định $0.5$). Phép so theo phần ở Công thức (1) là `cross` (lấy phần giống nhất); chế độ `fixed` thay bằng việc so *đúng* phần cùng tên.

### 5.4 Hai chiều ghép cặp

Hệ thống chạy được cả hai chiều trên cùng một mã nguồn (Hình 10):

- **Tìm CV cho tin tuyển dụng** (`jd2cv`): đầu vào là một tin tuyển dụng, kết quả là danh sách CV phù hợp nhất.
- **Tìm việc cho CV** (`cv2jd`): đầu vào là một CV, kết quả là danh sách tin tuyển dụng phù hợp nhất.




Chỉ khác nhau ở **vai trò**: tin tuyển dụng luôn là "bên yêu cầu", CV luôn là "bên đáp ứng".

**Figure 10:** Hai chiều ghép cặp dùng chung một cách tính điểm: một tin tuyển dụng đi tìm CV, và một CV đi tìm tin tuyển dụng.

Điểm giống nhau: công thức chấm điểm, cách lọc và cách giải thích *hoàn toàn như nhau*. Điểm khác nhau duy nhất là *vai trò* của hai tài liệu — tin tuyển dụng luôn đóng vai "bên yêu cầu", CV luôn đóng vai "bên đáp ứng" — và kho dữ liệu được tra cứu (CV hay tin tuyển dụng). Nhờ vậy chỉ cần viết và kiểm thử một lần.

## 6 Hợp đồng API (tóm tắt)

Mẫu phản hồi tiêu biểu của job (rút gọn):

| Method & Path | Trạng thái | Vai trò |
|---|---|---|
| `POST /api/v1/ai/analyze-cv` | 200 | Phân tích CV đồng bộ (trả JSON, không lưu) |
| `POST /internal/extract-cv` | 200 | Trích xuất + phân tích CV (nội bộ) |
| `POST /internal/extract-job` | 200 | Trích xuất + phân tích JD (nội bộ) |
| `POST /internal/index` | 200 | Nhúng + upsert vector cho danh sách payload |
| `POST /internal/index/delete` | 200 | Xoá vector theo id/type |
| `POST /internal/recommend` | 200 | Xếp hạng hybrid (kèm `explain`) |
| `POST /internal/explain` | 200 | Giải thích 1 cặp (job, cv) bằng LLM |
| `POST /internal/rerank` | 200 | Chấm lại theo rubric 0–100 |
| `POST /internal/embeddings` | 200 | Sinh vector thô cho danh sách chuỗi |
| `POST /internal/jobs` | 202 | Tạo job xử lý file (multipart) → `job_id` |
| `GET /internal/jobs/{id}` | 200 | Trạng thái + tiến độ + cảnh báo |
| `DELETE /internal/jobs/{id}` | 200 | Hủy job chưa xong |
| `POST /internal/jobs/{id}/retry` | 200 | Đưa job `FAILED` về `PENDING` |
| `GET /internal/jobs/stats` | 200 | Số job theo trạng thái, hàng đợi, phân vị thời gian |
| `POST /internal/jobs/reconcile` | 200 | Gỡ job kẹt (idempotent) |
| `GET /healthz`, `GET /readyz` | 200 | Sống/khỏe (ping Ollama, độ sâu hàng đợi) |

**Table 1:** Các endpoint của AI service. Mọi đường `/internal/*` đều bắt buộc `X-Internal-Api-Key` khi bật `INTERNAL_API_KEY_REQUIRED=1`.

```
{
  "job_id": "j_9f2c...", "status": "PROCESSING",
  "document_type": "cv", "document_id": "1234", "version": 3,
  "step": "PARSING", "progress": 55,
  "attempt_count": 1, "warnings": [],
  "duration_ms": null, "result": null
}
```

# 7 Cấu hình & công nghệ

| Nhóm | Thành phần | Lựa chọn | Lý do |
|---|---|---|---|
| Web frontend | Framework | Next.js | (repo riêng) |
| API nghiệp vụ | Backend | NestJS + Prisma | (repo riêng) |
| AI service | Ngôn ngữ | Python 3.13 + FastAPI | Async, OpenAPI tự sinh |
| LLM | Mô hình | `qwen3:4b` (Ollama) | Không mất dữ liệu ra ngoài |
| Embedding | Mô hình | `BAAI/bge-m3` (1024 chiều, CPU) | Đa ngôn ngữ, nhiều chức năng |
| Vector store | Cơ sở | ChromaDB persistent (HNSW, cosine) | Nhúng thư viện, đơn giản |
| Job store | Cơ sở | SQLite (WAL) | PG đang chỉ đọc; interface để chuyển Postgres |
| Từ khóa | Thuật toán | BM25 (`rank_bm25`) | Baseline kinh điển, giải thích được |
| Kiểm thử | Công cụ | pytest, ruff | 138 test tự động |

**Table 2:** Công nghệ chính của AI service.

Các biến môi trường quan trọng của hàng đợi (mặc định trong dấu ngoặc):

# 8 Kiểm thử & đánh giá

- **Kiểm thử tự động:** 138 test pytest bao phủ: trích xuất văn bản, chuẩn hoá payload, xếp hạng hybrid, giải thích/rerank (LLM giả lập), máy trạng thái job, worker (retry, backoff, job kẹt, hủy), hợp đồng API. Chroma dùng *ephemeral client* + `MockEmbedder` nên test chạy nhanh, không tải model thật.
- **Kiểm soát chất lượng mã:** `ruff` cho mã mới; tài liệu thiết kế (`docs/DESIGN_processing_queue.md`) được cập nhật đồng bộ với mã.

| Biến | Mặc định | Vai trò |
|---|---|---|
| OLLAMA_MODEL | – (bắt buộc) | Mô hình LLM dùng cho parse/explain/rerank |
| EMBED_MODEL | BAAI/bge-m3 | Mô hình sinh embedding |
| CHROMA_PATH | None | Nơi lưu Chroma; None = bộ nhớ (dùng cho test) |
| INTERNAL_API_KEY_REQUIRED | 0 | Bật = "fail-closed" khi chưa cấu hình key |
| PROCESSING_WORKER_ENABLED | 0 | Bật worker nền trong process FastAPI |
| PROCESSING_DB_PATH | data/processing.db | Đường dẫn SQLite của hàng đợi |
| PROCESSING_FILES_DIR | data/processing_files | Thư mục file tạm |
| JOB_MAX_ATTEMPTS | 3 | Số lượt thử tối đa cho lỗi tạm thời |
| JOB_STALE_SECONDS | 300 | Ngưỡng coi job PROCESSING là kẹt |
| JOB_RETRY_BASE_DELAY_S | 1.0 | Hệ số backoff $2^n \cdot \Delta$ |
| PROCESSING_SKIP_REEMBED | 1 | Bật bỏ qua nhúng lại khi nội dung không đổi |

**Table 3:** Cấu hình hàng đợi xử lý (xem đầy đủ tại apps/ai/.env.example).

- **Đánh giá chất lượng ghép cặp:** dùng bộ chỉ số chuẩn của truy hồi thông tin — Precision@k, Recall@k, nDCG@k, MRR, MAP — trên tập có nhãn (xem mục sau).

# 9 Đề xuất hướng nghiên cứu khoa học

## 9.1 Câu hỏi nghiên cứu

1. **RQ1 — Giá trị của việc tách theo section:** xếp hạng theo multi-vector từng section có tốt hơn nhúng toàn văn (single-vector) và BM25 thuần hay không?
2. **RQ2 — Pha trộn semantic – từ khóa:** giá trị $\alpha$ nào tối ưu (eq. 3) trên dữ liệu tiếng Việt/kép Việt – Anh?
3. **RQ3 — Vai trò của LLM ở cuối pipeline:** rerank theo rubric và giải thích evidence-grounded có cải thiện thứ tự xếp hạng và độ tin cậy của người dùng không (đo bằng LLM-as-judge và khảo sát người thật)?
4. **RQ4 — Chi phí – hiệu năng:** hàng đợi nền với bỏ qua nhúng lại (content_hash) tiết kiệm bao nhiêu thời gian/token so với xử lý đồng bộ và xử lý lại toàn bộ?

## 9.2 Ma trận thí nghiệm đề xuất

| Nhóm biến | Tùy chọn | Chỉ số |
|---|---|---|
| Phương pháp | BM25; single-vector; section-aware (mean/max); chunk max-sim | nDCG@10, MRR |
| Tham số | $\alpha \in \{0, 0.3, 0.5, 0.7, 1.0\}$; pool $P \in \{50, 100, 200\}$ | nDCG@10, độ trễ |
| Embedding | bge-m3; multilingual-e5-small; MiniLM đa ngôn ngữ | nDCG@10, thời gian nhúng |
| Tầng LLM | không rerank; rerank; rerank + explain | MRR, điểm LLM-as-judge (1–5) |
| Hàng đợi | đồng bộ; nền toàn bộ; nền + skip re-embed | thời gian phản hồi, token/ job |

**Table 4:** Ma trận thí nghiệm (mỗi cấu hình chạy 3 lần, lấy trung bình).

## 9.3 Dữ liệu & quy trình

- Bộ dữ liệu công khai (Kaggle: *Resume Dataset*, *Job Descriptions*) và dữ liệu nội bộ đã có sẵn pipeline đồng bộ từ PostgreSQL sang Chroma; yêu cầu tối thiểu 500 cặp CV–JD có nhãn "phù hợp / không phù hợp".
- Chia $70/10/20$ (train/dev/test); tham số $\alpha$, pool, ngưỡng lọc ràng buộc được tinh chỉnh trên dev, chỉ chốt một lần trên test.
- Đánh giá con người: 3–5 người đánh giá top-10 gợi ý theo thang 1–5 cho *mức độ phù hợp* và *mức độ hữu ích của phần giải thích*.

### 9.4 Lộ trình 12 tuần (đề xuất)

| Tuần | Giai đoạn | Nội dung |
|---|---|---|
| 1–2 | Chuẩn bị | Đọc tài liệu, dựng bộ dữ liệu có nhãn, hoàn tất baseline BM25 |
| 3–5 | Thí nghiệm | Chạy section-aware vs single-vector (RQ1), tinh chỉnh $\alpha$ (RQ2) |
| 6–8 | LLM | Thí nghiệm rerank + explain, đánh giá LLM-as-judge (RQ3) |
| 9–10 | Hệ thống | Đo chi phí hàng đợi vs đồng bộ, đo độ trễ đầu-cuối (RQ4) |
| 11–12 | Viết | Viết báo cáo, vẽ biểu đồ, chuẩn bị slide thuyết trình |

**Table 5:** Lộ trình nghiên cứu đề xuất.

### 9.5 Rủi ro & biện pháp

| Rủi ro | Biện pháp |
|---|---|
| Thiếu dữ liệu có nhãn | Dùng dữ liệu công khai + gán nhãn thủ công theo mẫu rubric; dùng LLM-assisted labeling rồi kiểm tra chéo 20% |
| LLM chấm không nhất quán | Chạy 3 lần, tính đồng thuận (kappa); chốt prompt cố định |
| Ollama/BGE-M3 chậm trên CPU | Giữ worker 1 thread, đo riêng từng bước, dùng pool nhỏ; nếu cần chuyển sang GPU/API embedding |
| Kết quả không có ý nghĩa thống kê | Kiểm định Wilcoxon signed-rank giữa các cấu hình |

**Table 6:** Rủi ro dự kiến.

### 9.6 Tài liệu tham khảo chính


- Chen, J. et al. (2024). *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. ACL Findings.
- Robertson, S. & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in IR.
- Karpukhin, V. et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP.
- Nogueira, R. & Cho, K. (2019). *Passage Re-ranking with BERT*. arXiv:1901.04085.
- Zheng, L. et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS Datasets and Benchmarks.
- Tài liệu thiết kế nội bộ: `docs/DESIGN_processing_queue.md`, `docs/WORKFLOW.md`, `plan.md` (trong repo).


## Phụ lục: cấu trúc thư mục liên quan

```
section-aware-cv-job-matching/
├── apps/ai/                    # AI service (FastAPI)
│   ├── app/
│   │   ├── main.py             # lifespan, worker thread, /healthz, /readyz
│   │   ├── routes.py           # 17 endpoint (analyze-cv, /internal/*)
│   │   ├── schemas.py          # Pydantic models
│   │   ├── security.py         # API key + fail-closed
│   │   ├── text.py             # PDF/DOCX/OCR, magic bytes, làm sạch
│   │   ├── llm.py              # client Ollama (retry, timeout)
│   │   ├── embeddings.py       # BGE-M3 (+ MockEmbedder cho test)
│   │   ├── normalize.py        # chuẩn hoá CV/JD -> payload
```

```
│   │   ├── recommend.py         # xếp hạng hybrid, index, explain
│   │   ├── explain.py           # giải thích + rerank bằng LLM
│   │   ├── index_store.py       # payload JSONL cạnh Chroma
│   │   └── processing/          # hang doi nen
│   │       ├── store.py         # SQLite + may trang thai job
│   │       ├── pipeline.py      # 4 buoc EXTRACTING -> FINALIZING
│   │       └── worker.py        # thread nen, retry, backoff, stale
│   ├── tests/                   # 135 test pytest
│   └── .env.example
├── src/cvjd/                    # thu vien loi (chay duoc tu CLI)
│   ├── schema.py, io.py
│   ├── data/                    # payload, preprocessing
│   ├── models/                  # bm25, chunking, embedding, matching
│   ├── ranking/                 # engine, hybrid, structured, constraints
│   ├── retrieval/               # retriever
│   └── storage/                 # chroma, postgres, outbox
└── docs/
    ├── DESIGN_processing_queue.md
    ├── WORKFLOW.md
    ├── kien_truc_he_thong.tex   # tai lieu nay (chi \input so do)
    └── so_do/                   # 10 so do TikZ tach rieng
        ├── fig01_tong_the.tex … fig07_vector_theo_section.tex
        ├── fig08a_luong_day_du.tex       # luong day du: upload -> tra ket qua
        ├── fig08_luong_ghep_cap.tex      # 4 buoc trong phan ghep cap
        ├── fig09_hai_chieu.tex           # jd2cv vs cv2jd
        ├── build_png.py         # sinh PNG cho tung so do
        ├── png/                 # anh PNG da sinh (200 dpi)
        └── _build/              # PDF trung gian (xoa duoc)
```